

YAGA

YAML Advanced Generator for Admin panels

Point it at an existing database. Describe the dashboard in one YAML file.

Get back a self-contained Go admin application — offline to generate, offline to run.

v2.1.5 PostgreSQL · SQLite · MSSQL one Go binary no npm / sqlc / Tailwind

[docs/USER_GUIDE.md](#) · [docs/Lua-for-Yaga-guide.md](#) · [SPEC.md](#)

Why YAGA exists

The database is already designed — YAGA adds the admin behaviour on top of it.

- No hand-written UI code: CRUD screens, search, sort, filters, forms, pagination, CSV import/export are all generated.
- One file is the source of truth. `yaga.yaml` describes screens, queries, actions, policies and behaviour; everything is re-derivable.
- Introspection first: foreign keys become record pickers, label joins and master–detail wiring automatically.
- The output is a real Go application — its own `go.mod`, chi router, templ views — that you can read, extend and own.
- No network at runtime: the stylesheet and Chart.js are vendored into the generated project.
- No toolchain beyond Go — no Node/npm, no sqlc, no Tailwind binary.
- One config generates for PostgreSQL, SQLite and MSSQL: placeholders, quoting, LIKE and pagination are the generator's problem.
- Everything is reversible: change the YAML, regenerate, rebuild.

The loop

edit YAML → yaga generate → make → test

End-to-end: database to running dashboard

Six steps — and steps 3 to 6 are the loop you repeat for the life of the product.

1 Design the database

Primary keys, natural label columns, real foreign keys. FKs are the wiring yaga builds on.

2 yaga init --db DSN

Introspect the DB, create users/roles + admin login, write yaga.yaml with the captured schema: block.

3 edit / wedit / AI

Refine labels, columns, forms, filters, actions, hooks, navigation, pages, theme.

4 yaga generate

Emit ./admin — handlers, SQL, templ views, auth, assets. Fully offline: no DB connection.

5 cd admin && make

go mod tidy → go tool templ generate → go build. One command (build.ps1 on Windows).

6 Run and test

./admin --port 8080, log in, exercise the screens — then go back to step 3.

Schema evolution: `yaga init --db DSN --update` merges newly added tables into an existing yaga.yaml — every customisation is preserved, the `schema:` block is replaced, and tables that disappeared are marked `# ORPHANED` for manual review.

One binary, six commands

Hand-rolled stdlib flags from ./cmd/yaga — no cobra, no viper.

Command	What it does
yaga init --db DSN	Introspect an existing database, write yaga.yaml with the schema: block and an admin login. The only scaffold.
yaga edit	Interactive TUI editor (tview). With --prompt it becomes an AI-assisted edit.
yaga wedit	Browser editor: local HTTP server, embedded SPA, live preview, MCP endpoint.
yaga generate	Emit the admin application into --out. Fully offline: no DB, no sqlc.
yaga validate	Parse + validate. --fix auto-repairs known problems, --dry-run previews.
yaga version	Print the yaga version.

The DSN is a runtime concern — it lives in .ENV (0600) beside the binary, overridable with DATABASE_URL.

```
# install
go install github.com/MichalHerstus/\
  MichalHerstus/yaga/cmd/yaga@latest
```

```
# from a checkout
go build -o yaga ./cmd/yaga
```

```
yaga init  --db DSN -c yaga.yaml \
           --out ./admin --force
yaga generate -c yaga.yaml -o ./admin
yaga validate --fix --dry-run
yaga wedit --port 9090 --open
```

```
# global flags
-c/--config      -o/--out
-d/--db          -p/--admin-password
-f/--force       -v/--verbose
-s/--skip-plugins --update
```

Database introspection is the entry point

There is no empty template and no demo mode: a real schema is the foundation.

What it does

1. Connect and introspect tables, columns, PKs and foreign keys.
2. Create users/roles (admin/manager/user) + admin@admin.test with a one-time bcrypt password (or --admin-password).
3. Write yaga.yaml: a resource per table, auth, navigation.
4. Plus the captured schema: block — the sole schema source.

Type mapping

int / serial → integer	varchar / text → string
bool → boolean	timestamp / date → datetime
real / float / numeric → float	json / jsonb → json
bytea / blob → file	(hand-edit the schema block if needed)

- Driver detection from the DSN prefix:
 - postgres:// | postgresql:// → pgx
 - sqlserver:// | mssql:// → go-mssqldb
 - anything else (file path, :memory:) → modernc.org/sqlite
- Foreign keys are exploited, not just recorded:
 - relation fields with modal record pickers
 - {fk}_label list columns resolved by LEFT JOIN
 - automatic children: master–detail blocks
- Database views are captured with view: true → read-only resources.
- On MSSQL, identity columns act as the key and PascalCase names are handled;
 - table: / id_column: / id_type: overrides are emitted when the naming conventions do not match the real schema.

The top-level blocks of yaga.yaml

The authoritative schema is in SPEC.md; this is the map of what each block does.

Block	Function
version	Config schema version (any non-empty value).
panel	Identity, name, branding colors and logo, layout (sidebar, topbar, max content width), theme and fonts.
connections	Database connections — driver, DSN, pool settings. The first entry drives the generated app.
schema	The captured database schema (tables, PKs, views, columns, foreign keys) — the sole source for offline generation.
auth	Login table, identity and password fields, post-login redirect, optional rate limit.
navigation	Sidebar groups and items — resource lists, page routes, external links.
resources	CRUD entities: list / card / detail / form views, actions, hooks, policies, children, computed columns.
pages	Custom dashboard pages built from stat, stats_grid, chart, table, list and html widgets.
audit	Audit log of every create/update/delete/action; adds an AuditLog resource and a navigation group.
procedures	SQLite SQL-batch stored procedures for proc: on SQLite; ignored on postgres / mssql.
plugins	Generation-time plugins contributing resources, pages and hook source files.

schema: — the single source of truth

This is what makes yaga generate fully offline and deterministic.

```
schema:
  tables:
    - name: orders
      primary_key: id
      view: false
      columns:
        - {name: id,          type: integer}
        - {name: total,       type: float}
        - {name: status,      type: string}
        - {name: placed_at,   type: datetime}
      foreign_keys:
        - column: customer_id
          foreign_table: customers
          foreign_column: id
          label: name
```

- Written by `init --db`, fully hand-editable afterwards.
- Every list, card, detail, form and options query is derived from it — no SQL query files, no `sqlc`, no `schema.sql`.
- Identifiers are quoted at generation time, so keyword-named tables (sqlite `Order`, mssql `Zamestnanec`) work on every driver.
- `validate` and both editors run a reference pass: a resource table or column missing here is an error / warning, and the editor jumps to the row.
- Because it is captured, generation never opens a database connection — the same YAML produces the same app anywhere.

Golden rule

A well-designed database produces a well-behaved admin panel almost for free.

resources: — CRUD entities and their views

Each resource maps to one table and produces a Go package, a route group and its views.

- Identity: name (PascalCase → Go package, directory, URL segment), label, plus table / id_column / id_type overrides.
- list: columns with label, type, sortable, searchable and format; default_sort (a - prefix means desc), per_page, search.
- filter: a collapsible filter bar compiled at generation time into SQL (and / or, comparisons, contains, is_null, \$N params).
- card: grid or kanban board via kanban_field; columns × rows set the page size.
- detail: fields; form: create/update fields with validation, visibility, relations and file uploads.
- computed: read-only expression columns (date_diff, coalesce, round, now).
- children: an embedded lines table with Add Line / Edit / Delete.
- policies + actions: per-verb and per-button RBAC.

```
resources:
  - name: Order      # → package order, URL /admin/order
    label: Orders
    table: orders
    import_csv: true
    list:
      per_page: 20
      default_sort: -placed_at
      columns:
        - {name: id,      label: '#', sortable: false}
        - {name: customer_id_label, label: Customer}
        - {name: total, label: Total, format: money}
      filter:
        label: Filters
        where: status = $1 and total >= $2
    form:
      create:
        fields:
          - {name: customer_id, type: relation,
              options_value: id, options_label: name}
```

wedit — the web editor (and edit — the TUI)

Four ways to change the config, all validating the same way.

- yaga wedit starts a local HTTP server (default :9090, --port/--open) with an embedded vanilla-JS SPA — no npm, no build step.
- Tab editors: panel, connections, auth, navigation (collapsible tree with missing-reference markers), resources, pages.
- Validate screen runs the full validator plus the schema-reference pass, with a Fix button for known-fixable problems (in memory until Save).
- Preview renders the dashboard mock 1:1 with the generated markup — pages, resource lists, light and dark theme, seeded fake data.
- Raw YAML tab, plus the validate / fix / save / rev / events endpoints.
- Edits live in memory; Save writes the file (MCP save keeps a .bak).
- Live sync: an SSE stream plus a revision counter pushes agent and tab changes to every open browser; a stale tab is warned before it can clobber newer work.

yaga edit — terminal UI

A 3-pane tview editor covering every config section, with live validation.

Ctrl+S save · Ctrl+V validate · Ctrl+Q quit · Esc back

Ctrl+P opens cd-style path navigation —

/Resources/User/List/Columns,

../Columns, with Tab autocompletion; Ctrl+O goes home.

Where the state lives

The server holds ONE in-memory config behind an RWMutex.

Every mutating path — SPA PUT, MCP Commit, raw apply, fix — funnels

through replaceConfig, which bumps the revision counter. That is what

keeps the agent, the browser tabs and the CLI in agreement.

Connecting an AI agent to YAGA

A one-shot prompt edit, or a full MCP session against the running wedit server.

A. `yaga edit --prompt "..."`

Sends the current config to a model (OpenRouter by default, or a local LM Studio server with `--model lmstudio` and no API key).

The reply is merged back as a YAML fragment — only the changed sections — then re-parsed and re-validated. An invalid merge is retried once; on a second failure the original file is left untouched.

```
panel/name -> 'Order management'
resources/User/list/columns/email/label
  -> 'Primary email'
```

B. MCP over wedit (POST /mcp)

Model Context Protocol, Streamable HTTP, JSON-RPC 2.0, served by the

same wedit process that serves the browser editor.

Tools: `get_config`, `get_value`, `set_value`, `merge_yaml_fragment`, `validate`, `save`, `open`, `list_resources`, `list_navigation`, `add/remove_resource`, `add_column`, `add_field`, `add_nav_item`, `remove_nav_item`.

```
{ "mcp": { "yaga": {
  "type": "remote",
  "url": "http://localhost:9090/mcp" } } }
```

- Every mutating tool goes through `commitOrError`: `Parse + ValidateAll` first, so an invalid edit is rejected and never touches the in-memory config — the same contract the web editor uses.
- A successful edit propagates to every connected browser tab instantly, and `save` writes a `.bak` before the real file.

Actions — buttons that do things

Custom buttons on a resource, per record or in bulk, running your logic at request time.

- Route: POST /<panel>/<resource>/<id>/action/<name> — an unknown name is a 404.
- A body is exactly one of:
 - query: inline raw SQL, the record id bound as \$1
 - proc: a stored procedure — CALL on Postgres, EXEC on MSSQL, a named procedures: SQL batch on SQLite
 - script: an embedded Lua body (next slide)
- Presentation: label, icon, color, requires_confirmation.
- policy: adds per-button RBAC on the action and bulk routes.
- bulk: true renders row checkboxes and a toolbar; the loop runs inside ONE transaction, so a mid-batch failure rolls the whole thing back.
- With audit enabled, the action and its audit INSERT share that transaction.

```
actions:
  - name: mark_done
    label: "Mark done"
    icon: check
    color: success
    policy: admin|manager
    requires_confirmation: true
    query: "UPDATE orders SET status='done' WHERE id=$1"

  - name: archive
    label: Archive
    bulk: true
    proc: sp_archive_customer
```

```
procedures:      # SQLite "stored procedure"
  - name: sp_archive_customer
    sql: |
      UPDATE customers SET status='inactive'
      WHERE id = $1;
      INSERT INTO customer_log (customer_id, msg)
      VALUES ($1, 'archived');
```

Hooks — before and after every mutation

Attach code to the lifecycle of create, update, delete and actions.

- Attachment points: form.create, form.update, form.delete and every action —
 - each with before and after.
- Exactly one body per hook, of four kinds:
 - fn: the generator emits a compile-ready func Name(s *hooks.Scope) stub; you fill in the Go body — full power
 - sql: an inline statement run with the record id
 - proc: a stored procedure call
 - script: an embedded Lua body (next slide)
- Scope: ID, Table, Action, User, Role and Values. On create and update the
 - Values map is passed in and written back, so a before-hook can fill in defaults for fields the user left empty.
- before-create sees ID = 0; after-create gets the real id from a driver-aware
 - RETURNING / OUTPUT INSERTED capture. A hook error aborts with HTTP 500.

```
form:
  create:
    fields:
      - {name: status, type: select,
        options: {draft: Draft, sent: Sent}}
    hooks:
      before:
        - name: default_status
          script: |
            if ctx.values["status"] == nil then
              ctx.values["status"] = "draft"
            end
      after:
        - name: notify
          sql: "INSERT INTO notifications (target, msg)
              VALUES ($1, 'user created')"
```

script: — Lua at request time, sandboxed

YAGA embeds gopher-lua and runs your body as `run(ctx)` under a fixed 5 s timeout.

- Where: inside the generated dashboard, once per invocation. Exceeding the
 - deadline aborts the script and the request fails with HTTP 500.
- Sandbox: `SkipOpenLibs` plus `base` / `table` / `string` / `math` only — no `io`, no `os`,
 - no `debug`, no `coroutine`; `require` resolves preloaded `stdlib` modules only.
- Host API: `db.exec`, `db.query`, `db.query_one`, `abort(msg)` → 400 plus a visible
 - flash, and `log(msg)`. `?` placeholders bind positionally on SQLite and are auto-renumbered to `$N` on PostgreSQL and MSSQL.
- Mutually exclusive: a hook uses one of `fn` / `sql` / `proc` / `script`, an action one
 - of `query` / `proc` / `script` — enforced by the parser.
- Bulk script actions loop per selected id; a script action can force the audit
 - write into the same transaction.

```
actions:
  - name: flag_audit
    label: Flag
    script: |
      if ctx.role ~= "admin" then
        abort("Only admins can flag")
      end
      local row = db.query_one(
        "SELECT * FROM items WHERE id = ?", ctx.id)
      if row then
        db.exec("UPDATE items SET flag=1 WHERE id=?",
          ctx.id)
      end
```

```
ctx.id      row id (0 in a before-create hook)
ctx.table   the resource table name
ctx.action  create | update | delete | <name>
ctx.user    authenticated user
ctx.role    the user's role
ctx.values  form values, read/write (create+update)
```

make — compiling the generated dashboard

The generator writes a Makefile and a build.ps1; you never hand-assemble a build.

```
cd admin

make          # build: tidy → templ generate → go build

# individual targets
make templ    # go tool templ generate (.templ → *_templ.go)
make tidy     # go mod tidy
make run      # build, then ./admin --port 8080 --log full
make package  # release tar.gz: binary + static + .ENV
make clean

# Windows: no make, no gcc, no CGO
powershell -ExecutionPolicy Bypass -File .\build.ps1 build
```

Security defaults in the generated app

CSRF token on every POST (session-stored, hidden field or header)
 Session rotation on login; SESSION_SECRET ≥ 32 chars in production
 Login rate limiting, POST-only logout, cookie flags
 Upload extension + content-type allow-list, attachment-only downloads
 CSV formula-injection escaping; CSP, X-Frame-Options, nosniff headers
 Errors logged server-side, generic text returned to the browser

- go.mod declares tool github.com/a-h/templ/cmd/templ, so templ resolves through the Go toolchain — nothing to install by hand.
- Assets are vendored at generation time: a pre-built Tailwind stylesheet and Chart.js 4.4.1 land in static/ — no npm step, no CDN.
- The DSN is a runtime concern: .ENV (0600) beside the binary, overridable with the DATABASE_URL environment variable.
- SQLite is pure Go, so static cross-compiles work:
 - CGO_ENABLED=0 GOOS=windows GOARCH=amd64 go build -o admin.exe .
- Runtime flags: --port/-p, --log/-l (full | err), -h. The database is
 - sanity-checked before the port is bound, so a missing DB is a clear startup error rather than a half-running server holding the port.

Takeaways

The database is the base

Introspect a real schema and keep the captured schema: block as the only schema source — generation is offline, deterministic and repeatable.

One YAML, many surfaces

CLI, TUI, web editor and MCP all read and write the same in-memory config, so the agent, the browser tabs and the CLI never disagree.

Behaviour without boilerplate

Actions and hooks in SQL, stored procedures, Go or sandboxed Lua cover the logic that CRUD builders cannot express.

Ships as ordinary Go

make produces a binary you can read, extend, cross-compile and deploy — with no npm, no sqlc and no network at runtime.

Next: `yaga init --db <dsn>` → `yaga wedit` → `yaga generate` → `make` docs/USER_GUIDE.md · docs/Lua-for-Yaga-guide.md · SPEC.md